

PTOSS-2

Testes Unitários — Guia Completo

FGA0314 – Testes de Software (2026.1) | Baserow – Web Frontend

Guia orientado para a **Seção 1 (Caixa-Preta + Caixa-Branca)** e **Seção 2 (TDD)** do trabalho.
Focado nos 6 métodos do **web-frontend** (Vue.js/Nuxt.js) do Baserow.

Método	Arquivo	Técnica
isValidEmail()	modules/core/validators.js	■ Caixa Preta
truncateText()	modules/core/utils/string.js	■ Caixa Preta
getColorClass()	modules/core/utils/color.js	■ Caixa Branca
matchesFilter()	modules/database/viewFilters.js	■ Caixa Branca
highlightSearchTerm()	utils/search.js (NOVA)	■■■■ TDD
buildFilterChain()	utils/filterChain.js (NOVA)	■■■■ TDD

Introdução e Contexto

Este guia foi elaborado para equipes sem experiência prévia em testes de software. Cada seção descreve passo a passo o que fazer, como fazer, como executar, e o que documentar ao final — tudo alinhado ao que o enunciado do PTOSS-2 exige.

Sobre o Projeto — Baserow (Web Frontend)

O Baserow é uma plataforma open-source de banco de dados no-code construída com **Nuxt.js + Vue.js** no frontend e Django no backend. O código-fonte do frontend está em **web-frontend/** e os testes unitários ficam em **web-frontend/test/unit/**. O framework de testes utilizado é o **Jest** em conjunto com o **Vue Test Utils**.

Configuração do Ambiente (faça isso UMA vez)

1

Clone o fork do grupo

```
git clone https://github.com/Gotc2607/Os-Pi--baserow.git && cd Os-Pi--baserow
```

2

Instale as dependências do frontend

```
cd web-frontend && yarn install
```

3

Execute os testes existentes para verificar

```
yarn test (todos devem passar — se der erro, veja o README)
```

4

Crie sua branch de trabalho

```
git checkout -b feat/testes-[seu-nome]
```

5

Onde ficam os testes?

Arquivos de teste ficam em **web-frontend/test/unit/** com extensão **.spec.js**

Como rodar apenas UM teste

```
yarn test --testPathPattern=<nome-do-arquivo>
```

Exemplo: `yarn test --testPathPattern=isValidEmail`

Para ver a cobertura: `yarn test --coverage`

SEÇÃO 1 — Testes Caixa-Preta e Caixa-Branca

O que são os dois tipos de teste?

Caixa-Preta (Black-Box): você *não olha* para o código-fonte. Testa apenas com base no que a função promete fazer (sua especificação). Técnicas: Particionamento de Equivalência (EP) e Análise de Valor Limite (BVA).

Caixa-Branca (White-Box): você *lê o código* e cria testes que garantam que cada caminho (if/else, condições) seja executado. Técnicas: Branch Coverage e MC/DC (Modified Condition/Decision Coverage).

Método 1 — isValidEmail(email)

■ CAIXA-PRETA

Arquivo: web-frontend/modules/core/validators.js

O que é essa função?

A função **isValidEmail(email)** recebe uma string e retorna **true** se for um endereço de e-mail válido, ou **false** caso contrário. É usada em formulários de cadastro e convite de usuários no Baserow.

Código da função (leia mas não altere)

```
// Arquivo: web-frontend/modules/core/validators.js
export function isValidEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  if (!email || typeof email !== 'string') {
    return false
  }
  return emailRegex.test(email.trim())
}
```

Passo a Passo — Como Desenvolver os Testes

1	Entenda a especificação Leia o código acima e entenda: o que faz a função? Quais entradas produzem true e quais produzem false?
2	Identifique as classes de equivalência (EP) Agrupe as entradas em classes: e-mails válidos, sem @, sem domínio, vazio, null, com espaços, etc.
3	Identifique os valores limite (BVA) Pense nos extremos: string vazia, 1 caractere, e-mail com exatamente o mínimo de partes.
4	Crie o arquivo de teste Crie: web-frontend/test/unit/validators/isValidEmail.spec.js
5	Escreva os testes com Jest Use describe/test/expect. Cada caso deve ter um nome claro que descreva o cenário.
6	Execute e verifique yarn test --testPathPattern=isValidEmail — todos os testes devem passar (verde).

Particionamento de Equivalência (EP)

Classe	Exemplo de entrada	Resultado esperado	Tipo
E-mail formato correto	usuario@dominio.com	true	Válida
E-mail com subdomínio	a@b.c.org	true	Válida
Sem arroba (@)	usuariodominio.com	false	Inválida

Como Executar e Verificar

Resultado esperado: **9 tests passed**, nenhuma linha vermelha.

- **Técnica utilizada:** Caixa-Preta (EP + BVA)
- **Justificativa dos casos:** para cada test(), explique a qual classe/limite pertence
- **Tabelas EP e BVA:** inclua as tabelas acima no relatório técnico
- **Evidência de cobertura:** screenshot do terminal com 'X tests passed'
- **Rastreabilidade:** mapeie EP-01 → test 'EP-01: e-mail válido retorna true', etc.
- **Discussão:** mencione que caixa-preta não revelou cobertura de branches — isso será complementado com caixa-branca em outro método.

Método 2 — truncateText(text, maxLength)

■ CAIXA-PRETA

Arquivo: web-frontend/modules/core/utils/string.js

O que é essa função?

A função **truncateText(text, maxLength)** recebe um texto e um comprimento máximo e retorna o texto truncado com '...' se ultrapassar o limite. É usada em rótulos de campos, nomes de tabelas e outros elementos da UI onde o espaço é limitado.

Código da função (leia mas não altere)

```
// Arquivo: web-frontend/modules/core/utils/string.js
export function truncateText(text, maxLength) {
  if (!text || typeof text !== 'string') {
    return ''
  }
  if (maxLength <= 0) {
    return ''
  }
  if (text.length <= maxLength) {
    return text
  }
  return text.substring(0, maxLength) + '...'
}
```

Passo a Passo

1	Entenda o contrato da função Para texto menor que maxLength: retorna o texto inteiro. Para texto maior: retorna os primeiros maxLength caracteres + '...'. Para entrada inválida: retorna string vazia.
2	Monte as classes de equivalência Pense em: texto normal (< limite), texto exato (= limite), texto longo (> limite), string vazia, null, maxLength=0, maxLength negativo.
3	Defina os valores limite Teste com maxLength = 1, texto com length = maxLength, text.length = maxLength + 1.
4	Crie o arquivo web-frontend/test/unit/utils/truncateText.spec.js
5	Escreva e execute yarn test --testPathPattern=truncateText --verbose

Particionamento de Equivalência (EP)

Classe	Exemplo de entrada	Resultado esperado	Tipo
--------	--------------------	--------------------	------


```

})
test('EP-07: maxLength negativo retorna string vazia', () => {
  expect(truncateText('Texto', -5)).toBe('')
})
// ■■ Análise de Valor Limite
test('BVA-01: texto com length == maxLength não trunca (limite exato)', () => {
  expect(truncateText('abc', 3)).toBe('abc')
})
test('BVA-02: texto com length == maxLength + 1 trunca', () => {
  expect(truncateText('abcd', 3)).toBe('abc...')
})
test('BVA-03: texto com length == maxLength - 1 não trunca', () => {
  expect(truncateText('ab', 3)).toBe('ab')
})
test('BVA-04: maxLength == 1 e texto de 2 chars trunca', () => {
  expect(truncateText('ab', 1)).toBe('a...')
})
})

```

O que Documentar

Checklist de documentação para este método

- **Técnica:** Caixa-Preta (EP + BVA)
- **Tabelas EP e BVA** preenchidas
- **Justificativa:** por que cada caso foi escolhido (qual classe ou limite representa)
- **Screenshot** do resultado 'X tests passed' no terminal
- **Discussão de integração:** mencione que a abordagem caixa-preta por si só não garante que todas as branches foram executadas — seria preciso caixa-branca para confirmar cobertura estrutural total.

Método 3 — getColorClass(color)

■ CAIXA-BRANCA

Arquivo: web-frontend/modules/core/utils/color.js

O que é essa função?

A função **getColorClass(color)** recebe uma string de cor e retorna a classe CSS correspondente para estilizar elementos da interface. Ela possui múltiplas branches (if/else if/else) que precisam ser todas cobertas.

Código da função — leia com atenção!

```
// Arquivo: web-frontend/modules/core/utils/color.js
export function getColorClass(color) {
  if (!color || typeof color !== 'string') {    // Branch A: entrada inválida
    return ''
  }
  const trimmed = color.trim().toLowerCase()
  if (trimmed === 'red') {                      // Branch B: vermelho
    return 'color--red'
  } else if (trimmed === 'blue') {             // Branch C: azul
    return 'color--blue'
  } else if (trimmed === 'green') {            // Branch D: verde
    return 'color--green'
  } else if (trimmed.startsWith('#')) {        // Branch E: hex
    return 'color--custom'
  } else {                                     // Branch F: desconhecido
    return 'color--default'
  }
}
```

Análise das Branches — FAÇA ISSO ANTES DE ESCREVER OS TESTES

Na caixa-branca, você analisa o fluxo do código e mapeia cada decisão. Abaixo está a análise de cada branch deste código:

Condição	Entrada	Saída esperada	Branch coberto
Entrada null/undefined/número	null	" (string vazia)	Branch A
color === 'red'	'red'	'color--red'	Branch B
color === 'blue'	'blue'	'color--blue'	Branch C
color === 'green'	'green'	'color--green'	Branch D
color começa com '#'	'#FF0000'	'color--custom'	Branch E
nenhuma das anteriores	'yellow'	'color--default'	Branch F
color com espaços extras	' Red '	'color--red'	Branch B (trim)
color em maiúsculas	'BLUE'	'color--blue'	Branch C (lower)

Passo a Passo

1	Leia o código-fonte Abra web-frontend/modules/core/utils/color.js e entenda cada if/else if/else.
2	Desenhe o grafo de fluxo Num papel, desenhe os caminhos: A → retorna vazio; B → retorna red; C → azul; etc.
3	Para cada branch, crie um caso de teste Cada if/else deve ter pelo menos um teste que o execute.
4	Para MC/DC: teste combinações de condições compostas Se houver '&&' ou ' ', crie testes que invertam cada condição individualmente.
5	Crie: web-frontend/test/unit/utils/getColorClass.spec.js Execute: yarn test --testPathPattern=getColorClass --coverage
6	Verifique a cobertura No relatório de cobertura, confirme que 'Branches: 100%' para esta função.

Código do Arquivo de Teste

```
// web-frontend/test/unit/utils/getColorClass.spec.js
import { getColorClass } from '@baserow/modules/core/utils/color'
describe('getColorClass – Caixa Branca (Branch + MC/DC)', () => {
  // Branch A – entrada inválida (null/undefined/não-string)
  test('[Branch A] null retorna string vazia', () => {
    expect(getColorClass(null)).toBe('')
  })
  test('[Branch A] número retorna string vazia', () => {
    expect(getColorClass(42)).toBe('')
  })
  // Branch B – red
  test('[Branch B] "red" retorna "color--red"', () => {
    expect(getColorClass('red')).toBe('color--red')
  })
  // Branch B com variação – trim + lowercase (cobre .trim().toLowerCase())
  test('[Branch B] " Red " (espaços e maiúsculas) retorna "color--red"', () => {
    expect(getColorClass(' Red ')).toBe('color--red')
  })
  // Branch C – blue
  test('[Branch C] "blue" retorna "color--blue"', () => {
    expect(getColorClass('blue')).toBe('color--blue')
  })
  test('[Branch C] "BLUE" retorna "color--blue"', () => {
    expect(getColorClass('BLUE')).toBe('color--blue')
  })
  // Branch D – green
  test('[Branch D] "green" retorna "color--green"', () => {
    expect(getColorClass('green')).toBe('color--green')
  })
  // Branch E – hex color
  test('[Branch E] "#FF0000" retorna "color--custom"', () => {
    expect(getColorClass('#FF0000')).toBe('color--custom')
  })
})
```

```

})
test('[Branch E] "#abc" retorna "color--custom"', () => {
  expect(getColorClass('#abc')).toBe('color--custom')
})
// Branch F – cor desconhecida
test('[Branch F] "yellow" retorna "color--default"', () => {
  expect(getColorClass('yellow')).toBe('color--default')
})
test('[Branch F] "purple" retorna "color--default"', () => {
  expect(getColorClass('purple')).toBe('color--default')
})
})
})

```

Como Ver a Cobertura

```

# Execute com flag de cobertura:
yarn test --testPathPattern=getColorClass --coverage
# No terminal, procure a linha de 'color.js':
# File          | % Stmts | % Branch | % Funcs | % Lines
# color.js      |    100   |    100   |    100   |    100
# Se "Branch" for 100% → todas as branches foram cobertas ■

```

O que Documentar

Checklist de documentação para este método

- **Técnica:** Caixa-Branca (Branch Coverage)
- **Grafo de fluxo** (pode ser um diagrama simples ou tabela de branches)
- **Tabela de branches** acima, mapeando qual teste cobre qual branch
- **Screenshot do relatório de cobertura** mostrando 'Branches: 100%'
- **Discussão de integração:** como caixa-branca revelou casos que caixa-preta poderia perder (ex.: trim/lowercase são detalhes de implementação não visíveis na especificação).

Método 4 — matchesFilter(type, fieldValue, filterValue)

■ CAIXA-BRANCA (MC/DC)

Arquivo: web-frontend/modules/database/viewFilters.js

O que é essa função?

A função **matchesFilter(type, fieldValue, filterValue)** é responsável por determinar se um valor de campo satisfaz um determinado filtro de view. Ela possui condições compostas (com && e ||) que exigem a técnica **MC/DC**: cada condição individual deve ser capaz de, sozinha, mudar o resultado final.

Código da função — leia com atenção!

```
// Arquivo: web-frontend/modules/database/viewFilters.js
export function matchesFilter(type, fieldValue, filterValue) {
  // Condição composta com múltiplas sub-condições
  if (type === 'equal') {
    return fieldValue !== null &&           // C1
           fieldValue !== undefined &&      // C2
           String(fieldValue) === String(filterValue) // C3
  }
  if (type === 'contains') {
    if (fieldValue === null || fieldValue === undefined) { // C4 || C5
      return false
    }
    return String(fieldValue)
      .toLowerCase()
      .includes(String(filterValue).toLowerCase())
  }
  if (type === 'not_empty') {
    return fieldValue !== null &&           // C6
           fieldValue !== undefined &&      // C7
           fieldValue !== ''               // C8
  }
  return false // tipo desconhecido
}
```

O que é MC/DC?

MC/DC (Modified Condition/Decision Coverage) exige que para cada condição dentro de uma decisão composta, exista ao menos um caso de teste em que **aquela condição, e somente ela**, seja a responsável pela mudança do resultado final (true → false ou false → true).

Exemplo: na decisão **C1 && C2 && C3**, você precisa de testes que mostrem: 'quando só C1 é false, o resultado muda'; 'quando só C2 é false, o resultado muda'; 'quando só C3 é false, o resultado muda'.

Análise MC/DC — Decisão 'equal': C1 && C2 && C3

Caso	C1 (≠null)	C2 (≠undef)	C3 (===)	Resultado	Propósito
M1	T	T	T	true	Todos true — resultado true

O que Documentar

Checklist de documentação para este método

- **Técnica:** Caixa-Branca (Branch Coverage + MC/DC)
- **Tabela MC/DC** para cada decisão composta (C1&&C2;&&C3;, C4||C5, C6&&C7;&&C8;)
- **Justificativa MC/DC:** para cada linha da tabela, explique qual condição está sendo 'independentemente testada'
- **Screenshot de cobertura** com 'Branches: 100%'
- **Discussão:** mencione quais casos funcionais (caixa-preta) teriam sido suficientes e quais branches MC/DC adicionaram cobertura extra.

Método 5 — highlightSearchTerm(text, term, options)

■■■ TDD (Test-Driven Development)

Arquivo novo a criar: web-frontend/modules/core/utils/search.js

Arquivo de teste: web-frontend/test/unit/utils/highlightSearchTerm.spec.js

O que é essa funcionalidade?

O Baserow possui um campo de busca global que destaca o termo pesquisado nos resultados. A função **highlightSearchTerm(text, term, options)** recebe um texto, um termo de busca e opções (como caseSensitive), e retorna o texto com o termo envolvido por tags de destaque HTML (). Esta função NÃO EXISTE ainda no repositório — você vai criá-la usando TDD.

O que o TDD exige — ciclos Red / Green / Refactor

Fase	O que fazer	Verificação
■ RED	Escreva o teste ANTES de existir qualquer código. Execute: yarn test — o teste deve FALHAR. Isso prova que o teste realmente detectaria um bug.	Terminal mostra: 'FAIL' ou 'Cannot find module'. Isso é esperado!
■ GREEN	Escreva o mínimo de código para o teste passar. Não se preocupe com qualidade ainda. Execute: yarn test — agora deve passar.	Terminal mostra: 'PASS'. Todos os testes verdes.
■ REFACT OR	Melhore o código sem mudar o comportamento. Remova duplicações, melhore nomes, simplifique lógica. Execute: yarn test — deve continuar passando.	Código mais limpo, testes ainda passando.

Especificação (leia antes de escrever os testes)

Comportamento esperado de highlightSearchTerm

- Recebe: **text** (string), **term** (string), **options** (objeto opcional)
- **options.caseSensitive** (boolean, padrão: false)
- **options.tag** (string, padrão: 'mark')
- Retorna: texto com todas as ocorrências do term envolvidas pela tag HTML
- Exemplo: highlightSearchTerm('Olá Baserow', 'baserow') → 'Olá <mark>Baserow</mark>'
- Se text ou term for vazio/null: retorna o text original sem modificações
- Deve substituir TODAS as ocorrências (global)
- Por padrão, a busca é case-insensitive
- options.tag permite usar outra tag, ex: 'strong' → termo

Passo a Passo TDD

1	CRIE o arquivo de teste (vazio por enquanto) web-frontend/test/unit/utils/highlightSearchTerm.spec.js — só com import e describe vazio
2	■ RED: Escreva o PRIMEIRO teste e execute Escreva apenas o teste mais básico (uma ocorrência simples). yarn test → deve FALHAR.
3	■ GREEN: Crie search.js com a implementação mínima Escreva o código mínimo em web-frontend/modules/core/utils/search.js para o teste passar.
4	■ RED: Adicione o segundo teste (múltiplas ocorrências) e execute Novo teste deve falhar se a implementação ainda não suportar múltiplas ocorrências.
5	■ GREEN: Atualize a implementação Adicione suporte a múltiplas ocorrências. yarn test → todos passam.
6	Repita para: case-sensitive, null/vazio, tag customizada, caracteres especiais A cada novo comportamento: RED primeiro, depois GREEN.
7	■ REFACTOR: Melhore o código Renomeie variáveis, extraia constantes, simplifique regex. yarn test deve continuar verde.

Fase 1 — ■ RED: Primeiro teste (escreva ANTES do código)

```
// PASSO 1: Crie este arquivo SEM criar search.js ainda
// web-frontend/test/unit/utils/highlightSearchTerm.spec.js
import { highlightSearchTerm } from '@baserow/modules/core/utils/search'
describe('highlightSearchTerm', () => {
  test('substituí uma ocorrência simples por <mark>', () => {
    const result = highlightSearchTerm('Olá Baserow', 'Baserow')
    expect(result).toBe('Olá <mark>Baserow</mark>')
  })
})
// Execute: yarn test --testPathPattern=highlightSearchTerm
// ESPERADO: FAIL - "Cannot find module" - isso é o RED normal!
```

Fase 2 — ■ GREEN: Implementação mínima

```
// PASSO 2: Crie web-frontend/modules/core/utils/search.js
export function highlightSearchTerm(text, term, options = {}) {
  if (!text || !term) return text || ''
  const tag = options.tag || 'mark'
  const flags = options.caseSensitive ? 'g' : 'gi'
  const escaped = term.replace(/[\.*+?^$(){}|\[\]\\"/g, '\\$&')
  const regex = new RegExp(escaped, flags)
  return text.replace(regex, (match) => `<${tag}>${match}</${tag}>`)
}
// Execute: yarn test → PASS (verde!) ■
```

Fase 3 — Adicione mais testes (RED → GREEN para cada um)

```
// Continue adicionando testes ao arquivo .spec.js
// Adicione UM por vez, execute, veja RED, depois garanta GREEN
```

```
// Múltiplas ocorrências
test('substitui TODAS as ocorrências', () => {
  const result = highlightSearchTerm('oi oi oi', 'oi')
  expect(result).toBe('<mark>oi</mark> <mark>oi</mark> <mark>oi</mark>')
})
// Case-insensitive por padrão
test('é case-insensitive por padrão', () => {
  const result = highlightSearchTerm('Baserow baserow BASEROW', 'baserow')
  expect(result).toBe(
    '<mark>Baserow</mark> <mark>baserow</mark> <mark>BASEROW</mark>'
  )
})
// Case-sensitive quando especificado
test('respeita caseSensitive:true', () => {
  const result = highlightSearchTerm('Hello HELLO hello', 'hello',
    { caseSensitive: true })
  expect(result).toBe('Hello HELLO <mark>hello</mark>')
})
// Tag customizada
test('usa tag customizada', () => {
  const result = highlightSearchTerm('Baserow', 'Baserow', { tag: 'strong' })
  expect(result).toBe('<strong>Baserow</strong>')
})
// Entradas inválidas
test('text vazio retorna string vazia', () => {
  expect(highlightSearchTerm('', 'abc')).toBe('')
})
test('term vazio retorna text original', () => {
  expect(highlightSearchTerm('Olá mundo', '')).toBe('Olá mundo')
})
test('null text retorna string vazia', () => {
  expect(highlightSearchTerm(null, 'abc')).toBe('')
})
// Caracteres especiais no term (regex safety)
test('term com caracteres especiais de regex não quebra', () => {
  const result = highlightSearchTerm('preço: R$10.00', 'R$10.00')
  expect(result).toBe('preço: <mark>R$10.00</mark>')
})
```

O que Documentar no TDD

Checklist de documentação — TDD

- **Evolução dos testes:** screenshots do terminal em cada fase (RED → GREEN → REFACTOR)
- **Evolução da implementação:** mostre como o código em search.js foi crescendo
- **Refatorações:** descreva as melhorias feitas (ex.: escape de regex extraído, opções com default)
- **Dificuldades:** o que foi difícil de testar? (ex.: regex com caracteres especiais)
- **Benefícios observados:** como o TDD ajudou a definir a API da função antes de implementá-la
- **Histórico de commits:** ESSENCIAL — faça commits separados para cada ciclo:
 - 'test(search): RED — primeiro teste highlightSearchTerm'
 - 'feat(search): GREEN — implementação mínima'
 - 'refactor(search): melhoria do escape de regex'

Método 6 — buildFilterChain(filters, fields)

■■■ TDD — Funcionalidade Mais Complexa

Arquivo novo: web-frontend/modules/core/utils/filterChain.js

Arquivo de teste: web-frontend/test/unit/utils/buildFilterChain.spec.js

Por que esta funcionalidade?

O Baserow permite que usuários apliquem múltiplos filtros simultâneos em views. A lógica de composição desses filtros (com AND/OR entre eles) é um ponto crítico e complexo do sistema. A função **buildFilterChain(filters, fields)** recebe um array de filtros e uma lista de campos e retorna uma **função de filtragem** que pode ser aplicada a uma linha (row) de dados. Isso envolve closures, composição funcional e lógica condicional — ideal para TDD.

Especificação Completa

Comportamento esperado de buildFilterChain

- **filters**: array de objetos com {field, type, value, operator} onde operator é 'AND' ou 'OR'
- **fields**: array de objetos com {id, name, type} descrevendo os campos disponíveis
- **Retorna**: uma função (row) → boolean
- Se filters for vazio ou null: retorna função que sempre retorna true
- Operador padrão entre filtros: AND (todos devem ser satisfeitos)
- Se operator for 'OR': basta um filtro ser satisfeito
- Filtro com field inexistente nos fields: é ignorado
- Tipos suportados: 'equal', 'contains', 'not_empty', 'greater_than' (só numérico)
- Exemplo de uso:

```
const fn = buildFilterChain([  
  {field:'nome', type:'contains', value:'Jo', operator:'AND'}  
], fields)  
fn({nome: 'João'}) → true  
fn({nome: 'Maria'}) → false
```

Estratégia TDD — Ciclos Planejados

Esta função é mais complexa. Planeje 5 ciclos RED→GREEN progressivos:

Fase	O que fazer	Verificação
■ RED	Ciclo 1: Teste mais simples possível. buildFilterChain([]) retorna uma função. Ciclo 2: Filtro 'equal' simples com AND. Ciclo 3: Dois filtros AND — ambos devem ser verdadeiros. Ciclo 4: Filtro OR — qualquer um verdadeiro. Ciclo 5: Filtro com field inexistente (deve ser ignorado).	Cada novo teste falha antes da implementação existir ou ser atualizada.
■ GREEN	Para cada RED: implemente o mínimo em filterChain.js que faça o teste passar. Não implemente o próximo requisito ainda — só o atual.	yarn test --testPathPattern=buildFilterChain → testes passando até o ponto atual.

■ REFACT OR	Após todos os ciclos, refatore: extraia a lógica de matching para uma função auxiliar, melhore nomes, trate edge cases de forma centralizada.	yarn test → todos os testes continuam passando após a refatoração.
--------------------	---	--

Arquivo de Teste Completo (escreva ciclo por ciclo)

```
// web-frontend/test/unit/utils/buildFilterChain.spec.js
import { buildFilterChain } from '@baserow/modules/core/utils/filterChain'
const FIELDS = [
  { id: 'nome', name: 'Nome', type: 'text' },
  { id: 'idade', name: 'Idade', type: 'number' },
  { id: 'ativo', name: 'Ativo', type: 'boolean' },
]
describe('buildFilterChain', () => {
  // ■■ CICLO 1: Sem filtros
  test('[C1] filtros vazios retorna função que sempre retorna true', () => {
    const fn = buildFilterChain([], FIELDS)
    expect(typeof fn).toBe('function')
    expect(fn({ nome: 'qualquer' })).toBe(true)
  })
  test('[C1] null retorna função que sempre retorna true', () => {
    const fn = buildFilterChain(null, FIELDS)
    expect(fn({})).toBe(true)
  })
  // ■■ CICLO 2: Um filtro 'equal'
  test('[C2] filtro equal: linha que bate com o valor → true', () => {
    const filters = [{ field: 'nome', type: 'equal', value: 'João', operator: 'AND' }]
    const fn = buildFilterChain(filters, FIELDS)
    expect(fn({ nome: 'João', idade: 30 })).toBe(true)
  })
  test('[C2] filtro equal: linha que NÃO bate → false', () => {
    const filters = [{ field: 'nome', type: 'equal', value: 'João', operator: 'AND' }]
    const fn = buildFilterChain(filters, FIELDS)
    expect(fn({ nome: 'Maria', idade: 25 })).toBe(false)
  })
  // ■■ CICLO 3: Dois filtros AND
  test('[C3] AND: ambos satisfeitos → true', () => {
    const filters = [
      { field: 'nome', type: 'equal', value: 'João', operator: 'AND' },
      { field: 'idade', type: 'equal', value: '30', operator: 'AND' },
    ]
    const fn = buildFilterChain(filters, FIELDS)
    expect(fn({ nome: 'João', idade: '30' })).toBe(true)
  })
  test('[C3] AND: só um satisfeito → false', () => {
    const filters = [
      { field: 'nome', type: 'equal', value: 'João', operator: 'AND' },
      { field: 'idade', type: 'equal', value: '30', operator: 'AND' },
    ]
    const fn = buildFilterChain(filters, FIELDS)
    expect(fn({ nome: 'João', idade: '25' })).toBe(false)
  })
  // ■■ CICLO 4: Filtro OR
  test('[C4] OR: apenas o primeiro satisfeito → true', () => {
    const filters = [
      { field: 'nome', type: 'equal', value: 'João', operator: 'OR' },
      { field: 'idade', type: 'equal', value: '99', operator: 'OR' },
    ]
    const fn = buildFilterChain(filters, FIELDS)
  })
}
```

[illegible]

Implementação de Referência (escreva após os testes falharem)

```
// web-frontend/modules/core/utils/filterChain.js
// IMPORTANTE: crie este arquivo APENAS após os testes estarem escritos e falhando
function applyFilter(filter, row) {
  const value = row[filter.field]
  switch (filter.type) {
    case 'equal':
      return value !== null && value !== undefined &&
        String(value) === String(filter.value)
    case 'contains':
      if (value === null || value === undefined) return false
      return String(value).toLowerCase().includes(
        String(filter.value).toLowerCase()
      )
    case 'not_empty':
      return value !== null && value !== undefined && value !== ''
    default:
      return true
  }
}

export function buildFilterChain(filters, fields) {
  if (!filters || filters.length === 0) {
    return () => true
  }
}
```

```

const fieldIds = new Set(fields.map((f) => f.id))
const validFilters = filters.filter((f) => fieldIds.has(f.field))
if (validFilters.length === 0) return () => true
// Determina operador global: se ALGUM for OR, usa OR; senão AND
const useOr = validFilters.some((f) => f.operator === 'OR')
return (row) => {
  if (useOr) {
    return validFilters.some((f) => applyFilter(f, row))
  } else {
    return validFilters.every((f) => applyFilter(f, row))
  }
}
}
}

```

O que Documentar no TDD (Método 6)

Checklist de documentação — TDD buildFilterChain

■ Commits separados por ciclo (obrigatório para evidenciar o processo):

- 'test(filterChain): RED C1 — filtros vazios'
- 'feat(filterChain): GREEN C1 — retorna função passthrough'
- 'test(filterChain): RED C2 — filtro equal simples'
- 'feat(filterChain): GREEN C2 — aplica filtro equal'
- ... e assim por diante até o Refactor final

■ Screenshots do terminal mostrando FAIL (RED) e PASS (GREEN) para cada ciclo

■ Evolução do código de filterChain.js a cada ciclo GREEN

■ Refatorações realizadas: extração de applyFilter, uso de Set para fieldIds

■ Dificuldades: decidir operador AND vs OR global, tratar fields inexistentes

■ Benefícios: a API ficou clara desde o início por causa dos testes; bugs de AND/OR foram detectados antes de integrar com o restante do sistema.

Integração Caixa-Preta ↔ Caixa-Branca

O enunciado pede que vocês demonstrem como as duas abordagens se complementam. Abaixo estão os pontos que você DEVE discutir no relatório:

Ponto de Discussão	Exemplo concreto do projeto
Caixa-preta não cobriu branch estrutural	Em <code>getColorClass</code> , os testes EP não testaram <code>case-insensitive</code> (<code>toLowerCase</code>) pois isso é detalhe de implementação — só a caixa-branca revelou essa branch.
Caixa-branca confirmou cobertura de EP	Em <code>matchesFilter</code> , os casos EP-01..EP-07 já cobriam a maioria das branches. MC/DC adicionou os casos M2/M3 que EP não teria gerado naturalmente.
Req. funcional ajudou a criar cenário estrutural	A especificação de 'busca case-insensitive' em <code>highlightSearchTerm</code> levou a criar o branch de <code>case-sensitive=true</code> no código — que não existiria sem o requisito.
TDD definiu API antes da implementação	Em <code>buildFilterChain</code> , os testes definiram a assinatura (<code>filters</code> , <code>fields</code>) → <code>fn(row)</code> antes de qualquer linha de código ser escrita, evitando retrabalho.

Estrutura de Commits — Guia Geral

Padrão de commits para o repositório

Para testes Caixa-Preta/Branca:

```
git commit -m 'test(isValidEmail): EP e BVA — caixa-preta'
git commit -m 'test(truncateText): EP e BVA — caixa-preta'
git commit -m 'test(getColorClass): branch coverage — caixa-branca'
git commit -m 'test(matchesFilter): MC/DC — caixa-branca'
```

Para TDD (um commit por fase):

```
git commit -m 'test(search): RED — highlightSearchTerm básico'
git commit -m 'feat(search): GREEN — implementação mínima'
git commit -m 'refactor(search): REFACTOR — extração de escape'
```

■ ■ Nunca faça um único commit com tudo — o histórico é evidência do processo TDD.

Estrutura do Relatório Técnico

- 1. Introdução — o que é o Baserow e o objetivo dos testes
- 2. Descrição do projeto — web-frontend, Nuxt.js, Jest, localização dos arquivos
- 3. Planejamento dos testes — quais 6 métodos foram escolhidos e por quê
- 4. Descrição das técnicas — EP, BVA, Branch Coverage, MC/DC, TDD
- 5. Testes desenvolvidos — para cada método: tabelas, código, screenshots de execução

- 6. Métricas de cobertura — screenshot de 'yarn test --coverage' por método
- 7. Descrição do processo TDD — ciclos RED/GREEN/REFACTOR com evidências de commits
- 8. Análise crítica — discussão de complementaridade entre caixa-preta e caixa-branca
- 9. Conclusão — lições aprendidas, dificuldades, impacto do TDD

Repositório da equipe — checklist final

- Código-fonte original do Baserow (web-frontend/)
- Arquivos de teste criados (web-frontend/test/unit/.../*.spec.js)
- Arquivos novos para TDD (web-frontend/modules/core/utis/search.js e filterChain.js)
- Instruções de execução no README (como rodar yarn test e yarn test --coverage)
- Relatórios de cobertura (screenshots ou arquivos gerados pelo Jest)
- Histórico de commits evidenciando ciclos RED/GREEN/REFACTOR